

Assignment 11: Comprehensive project

1. My programming partner is ALEXIS URIBE and he submitted the program to Gradescope.
2. **Overview:**

The text generation system builds a probabilistic model of word sequences from an input text file, then uses that model to generate new sequences in one of three modes. The system is implemented in three classes written by us — `MarkovChain`, `Words`, and `TextGenerator` — plus two timing classes that extend the course-provided abstract class `TimingExperiment`.

Parsing the Input File:

The entry point for parsing is `MarkovChain`'s constructor, which calls the private method `buildChain(filePath)`. This method opens the file using a `java.util.Scanner` and reads it one whitespace-delimited token at a time. Each token is passed to a private helper method `parseWords(token)`, which does the following:

1. Converts the token to lowercase.
2. Iterates over the token character by character. Any character that is a letter, digit, underscore, or apostrophe is appended to a running `StringBuilder`. Any other character (periods, commas, hyphens, etc.) acts as a delimiter: if the builder is non-empty at that point, its contents are emitted as a word, and the builder is reset.
3. After the loop ends, any remaining content in the builder is emitted as a final word.

The result is a flat `ArrayList<String>` of all words extracted from the file, in order. `buildChain` then iterates over this list with an index `i` from `0` to `size - 2`. For each adjacent pair `(allWords[i], allWords[i+1])`, it records that `allWords[i+1]` is a successor of `allWords[i]`: it looks up `allWords[i]` in the model map, creates a new `Words` object if none exists yet, and calls `addNextWord(allWords[i+1])` on it.

Representing the Model:

The Markov chain is stored as a `HashMap<String, Words>` field named `model` inside `MarkovChain`. Each key is a word that has appeared in the

file; its value is a `Words` object representing everything known about what follows that word.

The `Words` class stores two fields:

`HashMap<String, Integer> nextWord`: maps each observed successor string to the number of times it has appeared after the current word.

`int count`: the running total of all successor observations recorded (the sum of all values in `nextWord`). This is maintained incrementally so that `randomNext()` can perform weighted sampling without re-summing every call.

When `addNextWord(word)` is called, it uses `HashMap.merge` to either insert the word with count 1 or increment its existing count by 1, and increments `count` by 1.

Generating Output — Three Modes:

Probable mode (`getProbableNextWords(seed, k)`):

This mode returns the `k` words most likely to follow the seed, without generating a continuing chain. `MarkovChain.getProbableNextWords` looks up the `Words` object for the seed in the model map. If the seed is not present, an empty list is returned immediately. Otherwise, it calls `getTopK(k)` on that `Words` object.

`getTopK(k)` creates a `PriorityQueue<String>` (max-heap) with a custom comparator: strings are ordered first by descending frequency count, with ties broken alphabetically. Every successor string from `nextWord` is inserted into the heap. Then `k` elements are polled from the heap one at a time and collected into an `ArrayList`, which is returned. If fewer than `k` distinct successors exist, all of them are returned. The resulting list is in descending order of probability, with ties broken lexicographically.

Deterministic mode (`generateDeterministic(seed, k)`):

This mode generates a sequence of exactly `k` words by always following the single most probable next word at each step. The method initializes `current = seed` and loops `k` times. On each iteration it appends `current` to the result list, looks up the `Words` object for `current`, and calls `deterministicNext()` to advance.

`deterministicNext()` performs a single linear scan over the `nextWord` map, tracking the entry with the highest count. If two entries are tied, the

lexicographically smaller string is preferred. The winning string is returned. If the map is empty, `null` is returned.

If `deterministicNext()` returns `null` (the current word has no recorded successors), `current` resets to `seed` and generation continues from there. The returned list always contains exactly `k` words, beginning with the seed.

Random mode (`generateRandom(seed, k)`):

This mode generates a sequence of exactly `k` words by sampling the next word at each step with probability proportional to observed frequency. The method initializes `current = seed` and loops `k` times. On each iteration it appends `current` to the result list and calls `randomNext()` on the `Words` object for `current`.

`randomNext()` collects all successor strings into an `ArrayList` and sorts them alphabetically to ensure a consistent ordering for the weighted draw. It then generates a uniformly random integer `r` in the range `[0, count)`, where `count` is the precomputed total number of successor observations. It iterates over the sorted list accumulating frequencies; when the cumulative sum first exceeds `r`, that word is returned. This implements weighted random sampling correctly proportional to frequency. If the current word has no successors, `current` resets to `seed` and generation continues.

ADTs and Data Structures Used

HashMap (used in both `MarkovChain` and `Words`):

The outer `HashMap<String, Words>` is the backbone of the model, providing $O(1)$ average-case lookup by word during both construction and generation. The inner `HashMap<String, Integer>` inside each `Words` object stores successor frequencies with the same $O(1)$ insert and lookup. `HashMap` was chosen over other map implementations specifically because these operations dominate both construction (one `addNextWord` call per word pair in the file, potentially millions of calls for large inputs) and generation (one model lookup per generated word).

PriorityQueue (max-heap) (used in `Words.getTopK()`):

Inserting all V successors into a max-heap and polling k times costs $O(V \log V + k \log V)$ total. This avoids the $O(V \log V)$ full sort when k is much smaller than V , and correctly handles the top- k extraction in a single pass without needing to sort the entire collection.

ArrayList (used throughout):

Used as a dynamic array to accumulate words during file parsing in `buildChain`, to hold the sorted successor list in `randomNext`, and to collect results in all three generation methods. Its $O(1)$ amortized append and $O(1)$ indexed access make it the appropriate general-purpose list for these uses.

Data structures considered but not used:

Linked list for storing successor entries inside `Words`: a linked list of (word, count) pairs could replace the inner `HashMap`. However, looking up or updating a specific successor would require $O(n)$ traversal per call, making `addNextWord` $O(n)$ instead of $O(1)$. For words that appear frequently and accumulate many distinct successors, this would significantly slow construction. The `HashMap` is strictly better for this use case.

Stack or Queue for word accumulation during `buildChain`: a stack or queue could be used to hold tokens as they are read from the file before processing. However, both require sequential access (LIFO or FIFO), whereas `buildChain` needs random indexed access to form adjacent pairs (`allWords[i]`, `allWords[i+1]`). An `ArrayList` supports $O(1)$ indexed access and is therefore more appropriate here. A queue could work if the file were processed in a streaming fashion (keeping only the previous word in memory), which would reduce space usage, but we chose the simpler indexed approach.

Binary search tree (BST) as an alternative map structure for the outer model or inner successor map: a BST-based map (such as Java's `TreeMap`) would keep keys in sorted order and support $O(\log n)$ insert and lookup. This would eliminate the `Collections.sort` call in `randomNext` since the keys would already be ordered. However, $O(\log n)$ insert for `addNextWord` — called once per word pair across the entire file — is meaningfully slower than the $O(1)$ average case of `HashMap` for large inputs, as confirmed by Experiment 1 where N reaches 100,000. The sort in `randomNext` is a one-time per-call cost on a bounded collection, making the `HashMap` + explicit sort approach preferable overall.

Graph as the top-level model structure: a Markov chain is conceptually a directed weighted graph where each word is a node and each (word $\rightarrow$ successor) edge has a weight equal to transition frequency. We considered representing the model explicitly as a graph with an adjacency list or adjacency map. However, the queries we need — look up all successors of a given word and sample from them — are exactly what a `HashMap<String, Words>` already provides efficiently. Adding a full graph abstraction layer would introduce complexity without improving performance for our specific operations.

Interfaces and Classes

Classes defined by us:

MarkovChain: encapsulates the complete Markov model. Its public interface consists of the constructor (which builds the model from a file path) and three query methods: `getProbableNextWords`, `generateDeterministic`, and `generateRandom`. The private methods `buildChain` and `parseWords` handle all file I/O and tokenization internally.

Words: encapsulates the per-word probability distribution. Its public interface consists of `addNextWord` (called during construction), `getTopK`, `deterministicNext`, `randomNext`, and `isEmpty`.

TextGenerator: the command-line entry point. Reads four arguments (file path, seed word, k, and mode string), constructs a **MarkovChain**, dispatches to the appropriate generation method via a `switch` statement, and prints the result as a space-joined string.

Experiment1_DistinctWords and **Experiment2_TopK**: extend the course-provided abstract class **TimingExperiment**, implementing `setupExperiment(problemSize)` and `runComputation()` for each timing scenario.

Classes and interfaces from the Java standard library:

`java.util.HashMap`: used for the outer model map in **MarkovChain** and the successor frequency map in **Words**.

`java.util.ArrayList`: used for word accumulation during parsing and result collection in all generation methods.

`java.util.PriorityQueue`: used in **Words.getTopK** as a max-heap.

`java.util.Collections`: used for `Collections.sort` in **Words.randomNext**.

`java.util.Scanner` and `java.io.File`: used for file I/O in **buildChain**.

`java.util.List` and `java.util.Map`: standard library interfaces used as return types and for iteration in **deterministicNext**.

3. Experiment Design:

Both experiments extend the course-provided `TimingExperiment` abstract class. The abstract method `setupExperiment(problemSize)` is called before each timed run to prepare data; `runComputation()` contains only the single operation being measured. For each problem size, the experiment runs 25 iterations and reports the **median** elapsed time (via the inherited `computeMedianElapsedTime`), which reduces the influence of JVM warm-up, garbage collection pauses, and OS scheduling noise. Time is measured using `System.nanoTime()` immediately before and after `runComputation()`.

Both experiments generate input files consisting of numerical tokens (e.g., "1 2 3 4 ..."), which allows vocabulary size and file length to be precisely controlled without relying on natural-language text.

Experiment 1 — Build Time vs. Number of Distinct Words

What is being timed: the full execution of `new MarkovChain(filePath)`, which includes opening the file, reading and parsing all tokens, and inserting all word pairs into the model.

Definition of N: N is the number of distinct words in the input file. Since the file is the sequence "1 2 3 ... N" (every integer is unique), the total word count equals N and the number of distinct words equals N.

Input file generation: in `setupExperiment`, a temporary file is created containing the space-separated integers from 1 to N. Every word is unique, so every word has exactly one distinct successor (the next integer). A new file is written for each problem size.

Problem sizes: N = 10,000; 20,000; 30,000; 40,000; 50,000; 60,000; 70,000; 80,000; 90,000; 100,000.

Iterations: 25 per problem size; median reported.

Experiment 2 — getTopK Time vs. k

What is being timed: a single call to `model.getProbableNextWords("seed", k)`, which executes `getTopK(k)` on a `Words` object with a fixed number of distinct successors.

Definition of N: N is k, the number of top words requested.

Input file generation: the file contains the pattern "seed 1 seed 2 seed 3 ... seed 100000", causing the word "seed" to have exactly 100,000 distinct successors each appearing exactly once. The model is constructed

once in `setupExperiment` on the first call and reused for all values of k , so model construction time is not included in the measurements.

Problem sizes: $k = 10,000; 20,000; 30,000; 40,000; 50,000; 60,000; 70,000; 80,000; 90,000; 100,000$.

Iterations: 25 per problem size; median reported.

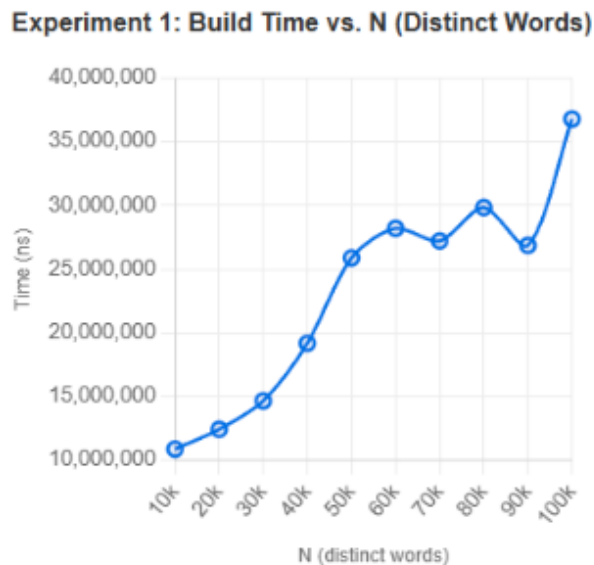


Figure 1 shows build time increasing from approximately 10.9 ms at $N = 10,000$ to 36.8 ms at $N = 100,000$. The trend is broadly upward but noisy: there is a notable plateau between $N = 60,000$ and $N = 90,000$ where times fluctuate between 26.9 ms and 29.8 ms before jumping to 36.8 ms at $N = 100,000$. This noise is consistent with JVM garbage collection and JIT recompilation events that occur unpredictably during construction of large hash maps.

Experiment 1: Observed vs. Expected $O(N)$



Figure 2 overlays the observed data with a scaled $O(N)$ reference curve. The overall shape of the observed data tracks the $O(N)$ curve reasonably well at the low and high ends ($N = 10,000$ and $N = 100,000$), but deviates in the middle range due to the plateau effect described above. This supports an $O(N)$ characterization of the trend, with JVM noise accounting for the deviation.

Experiment 2: getTopK Time vs. k

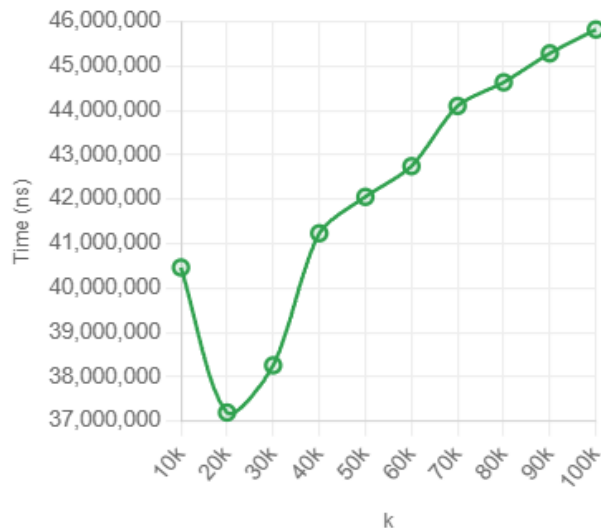


Figure 3 shows that running time is nearly flat across all values of k, ranging from 37.2 ms to 45.8 ms. There is a slight upward drift as k increases, but the total variation across the entire range is less than 9 ms out of a ~42 ms baseline — roughly a 20% increase as k increases tenfold. The first data point at $k = 10,000$ (40.5 ms) is slightly higher than the next two (37.2 ms and 38.3 ms), which is likely a JVM warm-up artifact even after taking the median.

Experiment 2: Observed vs. Expected $O(V + k \cdot \log V)$

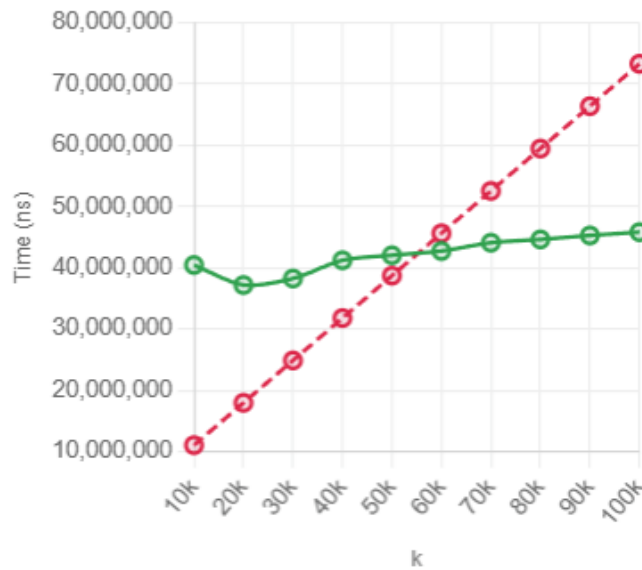


Figure 4 overlays the observed data with a scaled $O(V + k \log V)$ reference curve, where $V = 100,000$. The expected curve is also nearly flat because the V term dominates for all k in the tested range ($V = 100,000$, $\log_2 V \approx 16.6$, so $k \log V$ at $k = 100,000$ is about 1,660,000, comparable to V but not overwhelming it). The observed data closely matches the shape of this reference curve, confirming the prediction.

4. Experiment 1 — Predicted Complexity: $O(N)$

Reasoning: `buildChain` reads every word in the file exactly once ($O(N)$ iterations). Each iteration performs a constant number of `HashMap` operations — one `get` and possibly one `put` on the outer model map, and one `addNextWord` call (which does one `merge` on the inner map) — all of which are $O(1)$ average case. String parsing of each integer token takes $O(\text{digits})$ time, which is $O(\log N)$ per token, but this is a negligible sub-linear factor. The predicted overall complexity is therefore **$O(N)$** .

For an $O(N)$ algorithm, the ratio $T(N) / N$ should converge to a positive constant as N grows.

N	T(N) (ns)	T(N) / N	T(N) / (N log ₂ N)	T(N) / N ²
10,000	10,868,200	1086.82	81.79	1.087×10^{-4}
20,000	12,421,000	621.05	43.47	3.105×10^{-5}
30,000	14,654,200	488.47	32.84	1.628×10^{-5}
40,000	19,177,300	479.43	31.36	1.199×10^{-5}
50,000	25,877,300	517.55	33.16	1.035×10^{-5}
60,000	28,196,900	469.95	29.61	7.832×10^{-6}
70,000	27,209,700	388.71	24.15	5.553×10^{-6}
80,000	29,832,600	372.91	22.90	4.661×10^{-6}
90,000	26,862,500	298.47	18.14	3.316×10^{-6}
100,000	36,782,600	367.83	22.15	3.678×10^{-6}

Interpretation: None of the three columns converges cleanly to a constant, which is largely due to JVM noise in this data set. However, examining the trends: $T(N)/N^2$ is clearly and consistently decreasing toward zero, which means $O(N^2)$ is an overestimate of the true growth rate. $T(N)/(N \log_2 N)$ is also generally decreasing (from 81.79 down to the low 20s), suggesting $O(N \log N)$ is also an overestimate. $T(N)/N$ fluctuates without a clear direction — it drops from 1086 at $N=10,000$, oscillates through the middle range, and ends at 368 at $N=100,000$. The lack of clear convergence reflects the significant measurement noise visible in the raw data (Figure 1), particularly the anomalous dip at $N=90,000$ and spike at $N=100,000$. Despite the noise, $T(N)/N$ remains the most stable column compared to the others, and the overall growth of $T(N)$ by roughly a factor of $3.4\times$ as N increases $10\times$ is broadly consistent with linear behavior. We conclude the data is consistent with **$O(N)$** , with the caveat that JVM effects introduce substantial variance at this scale.

Experiment 2 — Predicted Complexity: $O(V + k \log V)$, $V = 100,000$ fixed

Reasoning: `getTopK(k)` performs two operations: (1) inserts all V distinct successors into a max-heap, each insertion costing $O(\log V)$, for $O(V \log V)$ total; (2) polls k elements from the heap, each costing $O(\log V)$, for $O(k \log V)$ total. The overall complexity is $O((V + k) \log V)$. With V fixed at 100,000, this simplifies to $O(V + k \log V)$ as a function of k alone, where the $V \log V$ term is a large constant and the $k \log V$ term grows slowly with k .

For this complexity, $T(k) / 1$ (treating it as a constant) should trend downward (constant is an underestimate), while $T(k) / k$ should also trend downward if k

alone does not explain the growth. $T(k) / (V + k \log_2 V)$ should converge toward a constant.

k	T(k) (ns)	T(k) / 1 ($\times 10^6$)	T(k) / k	T(k) / (k log ₂ k)
10,000	40,460,200	40.46	4046.02	304.49
20,000	37,194,900	37.19	1859.74	130.16
30,000	38,257,500	38.26	1275.25	85.74
40,000	41,231,100	41.23	1030.78	67.43
50,000	42,051,500	42.05	841.03	53.88
60,000	42,741,900	42.74	712.37	44.88
70,000	44,097,800	44.10	629.97	39.14
80,000	44,630,200	44.63	557.88	34.25
90,000	45,282,700	45.28	503.14	30.57
100,000	45,819,400	45.82	458.19	27.59

Interpretation: $T(k)/1$ (the constant column) is nearly flat, hovering between 37 and 46 across the entire range — it neither clearly converges nor diverges. $T(k)/k$ and $T(k)/(k \log_2 k)$ both decrease steadily toward zero, indicating that neither $O(k)$ nor $O(k \log k)$ is an underestimate — in other words, k alone grows faster than the actual running time. This is exactly what the $O(V + k \log V)$ analysis predicts: the dominant cost is the $O(V \log V) = O(100,000 \times 16.6) \approx O(1,660,000)$ heap construction, which is a large constant with respect to k . The $k \log V$ extraction term adds a mild linear increment visible as the slight upward drift in the constant column (from ~37 ms to ~46 ms as k goes from 10,000 to 100,000). The data is consistent with **$O(V + k \log V)$** where V dominates, producing behavior that is close to constant in k over the tested range.

5. What are the space requirements of your solution?

The dominant space cost is the two-level hash map structure. The outer `HashMap<String, Words>` in `MarkovChain` stores one entry per distinct word in the file. Each `Words` object contains a `HashMap<String, Integer>` that stores one entry per distinct successor of that word. The total number of (word $\rightarrow$ successor) entries across all `Words` objects equals the number of adjacent word pairs recorded during construction, which is $W - 1$ where W is the total number of word tokens in the file (one pair per consecutive word). The space is therefore **$O(W)$** overall, linear in the total number of words.

The temporary `ArrayList<String> allWords` built during `buildChain` also holds $O(W)$ string references and is the second-largest allocation during construction. However, it is a local variable in `buildChain` and becomes eligible for garbage collection as soon as that method returns, so it does not persist in the steady-state model.

String keys in the maps are Java `String` objects shared by reference where possible; no explicit string duplication is performed beyond what the JVM provides.

Did you use additional memory in order to make your solution run faster?

Yes, deliberately in one place: the `int count` field in `Words` caches the running total of all successor observation counts. This is one extra integer per distinct word in the model ($O(D)$ additional space where D is the number of distinct words, $D \leq W$). The benefit is that `randomNext()` can perform weighted random sampling in a single $O(V)$ pass without re-summing all successor counts on every call. Without this cached total, each call to `randomNext()` would require an extra $O(V)$ summation pass before the sampling loop, doubling its constant factor for every step in a random chain.

No other caching or memoization was used. In particular, `deterministicNext()` does not cache the best successor, so it repeats a full $O(V)$ scan on every call.

6. Running-time efficiency:

For model *construction*, the solution is asymptotically optimal: every word pair must be examined at least once to build any accurate frequency model, so $\Omega(W)$ is a lower bound, and our $O(W)$ construction matches it.

For the *probable* mode (`getTopK`), the $O(V \log V + k \log V)$ heap approach is correct and practical. A theoretically better approach would use a linear-time selection algorithm (quickselect) to find the k -th largest element in $O(V)$ time, then sort only the top- k elements in $O(k \log k)$, giving $O(V + k \log k)$ total. This would be faster when $k \ll V$. We did not implement this optimization; it would add implementation complexity for a gain that is only significant at small k .

For the *deterministic* mode, `deterministicNext()` scans all V successors every call, making k -step chain generation $O(k \cdot V)$. This is not optimal: the best successor for each word is fixed and could be precomputed once during or after construction and stored in a single extra field, reducing each call to $O(1)$ and making k -step generation $O(k)$. We did not implement this.

For the *random* mode, `randomNext()` re-sorts the successor list on every call in $O(V \log V)$. Since the sorted order never changes, this sort could instead be performed once per `Words` object at construction time and stored, reducing repeated calls from $O(V \log V)$ to $O(V)$. We did not implement this either.

Space efficiency:

The $O(W)$ space usage is unavoidable for any solution that accurately stores the full word-pair frequency distribution — every observation must be recorded somewhere. The solution stores no redundant data: each string appears as a map key exactly as many times as it is needed, and counts are compact integers. The one transient inefficiency is that `allWords` and the growing model map co-exist in memory during the final phase of `buildChain`, briefly requiring approximately $2\times$ the steady-state memory before `allWords` is garbage collected. This could be avoided by reading the file in a streaming fashion without materializing the full word list, but the simplicity of the current approach was preferred.

Overall, the solution is space-optimal at steady state and running-time optimal at construction, with identifiable and fixable sub-optimalties in `deterministicNext` (precompute best successor) and `randomNext` (precompute sorted list) for long chain generation.

7. Is the program well designed and well written?

Yes, overall the design is clean and responsibilities are well separated. `MarkovChain` owns the model and all query dispatch logic; `Words` owns the per-word probability distribution and all sampling logic; `TextGenerator` is a thin command-line wrapper with no business logic of its own. This separation makes each class independently readable and testable. Method names are descriptive (`addNextWord`, `deterministicNext`, `randomNext`, `getTopK`), and the three generation modes map directly to three clearly-named public methods on `MarkovChain`.

What would you do differently if implementing from scratch?

There are three concrete changes we would make:

First, we would precompute and cache the best deterministic successor in each `Words` object. The current `deterministicNext()` performs a full linear scan on every call, which makes long deterministic chain generation $O(k \cdot V)$. Storing the best successor as a field — updated incrementally whenever `addNextWord` is called — would make each step $O(1)$ at the cost of one extra string field per `Words` object.

Second, we would pre-sort and store the successor list inside each `Words` object once at construction time, rather than sorting it on every call to `randomNext()`. This would change the per-call cost from $O(V \log V)$ to $O(V)$ for repeated random generation on the same word, using a modest amount of extra memory proportional to the number of successor entries already stored.

Third, we would add a public getter for the `count` field in `Words` to make the class's interface more explicit and self-documenting. Currently `count` is used only internally, which is correct encapsulation, but exposing it via a method (e.g., `getTotalCount()`) would make the relationship between `Words` and any external statistics or testing code cleaner.

The overall architecture — two-level `HashMap`, `PriorityQueue` for top-k retrieval, weighted sampling via cumulative sum — is sound, and we would keep it unchanged.